ABSTRACT

The Sai Deepak Film Booking System is a full-featured, end-to-end cinema ticket reservation web application developed using HTML5, CSS3, Vanilla JavaScript (ES6+), PHP 7.4+, and MySQL 5.7+. The system replicates a professional cinema booking experience, enabling users to browse a curated catalogue of currently showing films, select preferred dates and showtimes from a dynamic date picker, interactively choose seats from a visual seat map, and complete ticket purchases through a streamlined three-step booking wizard.

The application is structured around a dual-mode operation strategy: it operates fully in the browser using localStorage as the data persistence layer when no server is present, and seamlessly upgrades to a persistent full-stack experience when deployed on a PHP/MySQL server. This design philosophy ensures the project can be demonstrated and evaluated without any server infrastructure while remaining production-deployable with minimal changes.

The project demonstrates key competencies in full-stack web development, relational database design, RESTful API architecture, and user interface design. It provides a solid, extensible foundation for real-world enhancements including payment gateway integration, real-time seat availability, email confirmations, and an administrative dashboard for theatre management.

Key Highlights:

- Three-step booking wizard: Movie Selection → Seat Selection → Payment Confirmation
- User authentication system with bcrypt password hashing (PHP backend) and localStorage fallback
- Interactive seat map with real-time seat state management (Available, Selected, Taken)
- Genre-based movie filtering across four categories: Action, Drama, Sci-Fi, and Horror
- Booking history management with cancellation support
- RESTful PHP API exposing endpoints for movies, registration, login, booking, and cancellation
- Cinematic dark-theme UI with Bebas Neue / Barlow typography and CSS grain overlays
- Fully responsive layout — adapts from desktop to mobile without loss of functionality
- Toast notification system for real-time user feedback.

Technology Stack Summary:

Layer	Technology	Purpose
Markup	HTML5	Page structure and semantic layout
Styling	CSS3 + Google Fonts	Dark-theme UI, animations, responsive grid
Client Logic	Vanilla JavaScript ES6+	DOM manipulation, API calls, state management
Server Logic	PHP 7.4+	REST API, authentication, database operations
Database	MySQL 5.7+	Users, movies, showtimes, bookings storage
HTTP Client	Fetch API	Async communication with api.php
Fallback Storage	localStorage	Offline data persistence

TABLE OF CONTENTS

Chapter / Title	Page No.
1. Introduction	5
1.1 Background	5
1.2 Objectives	6
1.3 Scope	7
1.4 Project Significance	8
1.5 Document Organization	9
2. System Requirements	10
2.1 Hardware Requirements	10
2.2 Software Requirements	11
2.3 Development Environment Setup	12
2.4 Deployment Requirements	13
3. System Architecture	14
3.1 Three-Tier Architecture	14
3.2 Data Flow Diagram	15
3.3 Component Interaction Model	16
3.4 Dual-Mode Operation Strategy	17
4. Modules Description	18
4.1 User Authentication Module	18
4.2 Movie Browsing Module	20
4.3 Booking Workflow Module	21
4.4 My Bookings Module	24
4.5 PHP Backend API Module	25
4.6 Notification Module	27
5. Database Design	28
5.1 Entity Relationship Overview	28
5.2 Table Descriptions	29
5.3 SQL Schema	32

Chapter / Title	Page No.
5.4 Database Normalization	33
6. File Structure	34
6.1 File Organization	34
6.2 File Roles and Responsibilities	35
7. User Interface Design	36
7.1 Design Philosophy	36
7.2 Home Page	37
7.3 Booking Page	38
7.4 Modals	39
7.5 Toast Notifications	40
7.6 Responsive Design	40
8. Functional Description	41
8.1 User Registration and Login	41
8.2 Booking Submission	42
8.3 Booking Cancellation	43
8.4 API Toggle & Graceful Degradation	43
9. Technology Stack	44
10. Testing	45
10.1 Testing Approach	45
10.2 Test Cases	46
10.3 Browser Compatibility	47
11. Limitations, Future Enhancements and UI Screen Shorts	48
12. Conclusion	49
13. References	50

1. INTRODUCTION

1.1 Background

The entertainment industry has undergone a profound transformation over the past two decades. The advent of digital technology and internet connectivity has fundamentally altered the way audiences access and consume cinematic experiences. What was once a process confined to physical box office queues has evolved into a seamless, highly automated online experience where users can browse films, compare showtimes, select preferred seats, and complete payment — all from the comfort of their homes or on the move using mobile devices.

Historically, cinema booking relied heavily on manual processes. Customers were required to visit the theatre, often in person and well in advance, to purchase tickets. This approach introduced multiple friction points including waiting queues, limited seat visibility, and no guarantee of availability. The introduction of telephone booking systems in the late twentieth century provided minor improvements but still required direct human interaction and offered no visual seat selection capability.

The emergence of web-based booking platforms changed the industry fundamentally. Platforms such as BookMyShow in India, Fandango in the United States, and Odeon Online in the United Kingdom demonstrated that online cinema ticketing could dramatically increase customer satisfaction while simultaneously improving operational efficiency for theatre managers. These platforms offer real-time seat availability, genre filtering, user accounts, booking history, and in many cases, integration with loyalty reward programmes and payment wallets.

The Sai Deepak Film Booking System is an academic project designed to replicate and study this modern cinema booking experience. Built entirely with standard, widely-supported web technologies — HTML5, CSS3, Vanilla JavaScript, PHP, and MySQL — it provides a fully functional booking workflow that demonstrates all core competencies required in modern full-stack web development. The project deliberately avoids heavyweight frameworks in order to ensure accessibility for developers at all levels and to minimise deployment complexity.

The name 'Sai Deepak' reflects a personalised, context-rooted identity for the system, positioning it as a cinema brand in its own right rather than a generic demonstration. The system's dark-theme, cinematically-inspired design language reinforces this branding throughout every user interaction.

1.2 Objectives

The project is guided by a set of clearly defined primary and secondary objectives that together shape the scope, architecture, and design of the system. These objectives are listed below in order of priority.

Primary Objectives:

1. To design and develop a fully responsive, visually compelling cinema booking interface that provides an experience comparable to professional ticketing platforms.
2. To implement a multi-step booking workflow covering three sequential stages: movie selection, interactive seat selection, and payment confirmation.
3. To develop a user authentication system supporting both registration and login, with secure password handling via bcrypt hashing on the server-side PHP backend.
4. To integrate a PHP backend REST API with a MySQL relational database for persistent data storage of users, movies, showtimes, and bookings.
5. To ensure the system degrades gracefully to browser localStorage-based operation when the PHP/MySQL server is unavailable, enabling demonstration without infrastructure.
6. To support complete booking lifecycle management including the creation, retrieval, and cancellation of bookings by authenticated users.

Secondary Objectives:

To apply responsive design principles using CSS media queries so the interface adapts correctly across desktop, tablet, and mobile screen sizes.

7. To implement genre-based movie filtering entirely on the client side without additional server requests.

8. To provide a toast notification system that delivers contextual feedback for all user actions including login, registration, booking confirmation, and errors.

9. To document the system thoroughly, covering architecture, database design, module descriptions, testing, and future enhancement pathways.

1.3 Scope

The scope of this project is defined by the functional boundaries of the system as built and tested. It is important to articulate both what the system does and what it explicitly excludes, as this framing assists evaluators and future developers in understanding the project's designed limits.

Within Scope:

- A web-based cinema booking interface accessible through standard modern browsers on any operating system.
- A catalogue of eight films across four genres: Action, Drama, Sci-Fi, and Horror.
- A seven-day rolling date picker for showtime selection.
- Five fixed showtime options per film per date.
- An interactive visual seat map with seven rows (A through G) and ten columns per row (70 seats total).
- A maximum of eight seat selections per booking session.
- User registration and login with session state managed through browser localStorage.
- A PHP REST API (api.php) serving six actions: register, login, movies, booking, bookings, cancel.
- A MySQL database schema with four tables: users, movies, showtimes, bookings.
- Booking history view and cancellation for authenticated users.
- A fully operational offline mode using localStorage when the PHP backend is disabled.

Outside Scope:

- Integration with any real payment gateway (Razorpay, Stripe, PayPal, etc.).
- Real-time dynamic seat availability based on concurrent bookings.
- Email confirmation or e-ticket generation via email services.
- An administrative dashboard for theatre management.
- Password recovery or two-factor authentication.
- Native mobile application (iOS or Android).
- Social media login integration.

1.4 Project Significance

The Sai Deepak Film Booking System holds significance across multiple dimensions — academic, technical, and practical.

From an academic perspective, the project brings together all major competencies of web development into a single cohesive deliverable. It requires the student to work across the presentation layer (HTML5, CSS3, JavaScript), the application layer (PHP), and the data layer (MySQL), thereby demonstrating full-stack proficiency in a way that isolated exercises cannot achieve. The deliberate avoidance of frameworks forces a deep understanding of core language features and raw API usage.

From a technical perspective, the project confronts several non-trivial engineering challenges. These include designing a RESTful API with consistent error handling and JSON response formatting, implementing a dual-mode data persistence strategy that switches transparently between server and browser storage, managing multi-step form state across a booking wizard without a client-side framework, and rendering an interactive seat map with complex state transitions (available, selected, taken) using pure JavaScript.

From a practical perspective, the completed system represents a production-ready prototype that, with the addition of a real payment gateway and email confirmation service, could be deployed as an operational cinema booking solution for a small to medium-sized theatre. Its flat file structure and minimal dependencies make deployment accessible even in shared hosting environments.

1.5 Document Organization

This report is structured into thirteen chapters. Chapter 2 specifies hardware and software requirements. Chapter 3 describes the system architecture including its three-tier design and data flow. Chapter 4 details each functional module. Chapter 5 presents the database design with entity relationships and table schemas. Chapter 6 documents the file structure. Chapter 7 describes the user interface design philosophy and layout. Chapter 8 explains functional behaviour. Chapter 9 summarises the technology stack. Chapter 10 presents the testing approach and results. Chapter 11 discusses limitations and future enhancement pathways. Chapter 12 provides a conclusion. Chapter 13 lists references.

2. SYSTEM REQUIREMENTS

This chapter specifies the hardware, software, and environmental requirements necessary to develop, deploy, and operate the Sai Deepak Film Booking System. Requirements are categorised into minimum and recommended tiers where applicable.

2.1 Hardware Requirements

The system is a web application and therefore its hardware requirements relate to the machine that hosts the web server (for full-stack deployment) and the device used by end users to access the booking interface (for client-side usage).

Server-Side (Hosting Machine) Hardware Requirements:

Component	Minimum Requirement	Recommended
Processor	Intel Core i3 or equivalent (1.5 GHz+)	Intel Core i5 / AMD Ryzen 5 (2.5 GHz+)
RAM	2 GB	4 GB or more
Storage	100 MB free disk space	1 GB SSD for future growth
Network	Broadband internet (for deployment)	Dedicated server with static IP
Display	N/A (headless server acceptable)	N/A

Client-Side (End User Device) Hardware Requirements:

Component	Minimum Requirement
Processor	Any modern dual-core processor (1.0 GHz+)
RAM	1 GB
Display	1024 x 768 resolution or higher
Network	Broadband or mobile data connection (for full-stack); any device for offline mode
Input	Keyboard and mouse (desktop); touchscreen (mobile)

2.2 Software Requirements

The following software is required for developing and deploying the system. Client-side usage requires only a modern web browser; no additional software installation is needed by end users.

Software	Version / Details	Role
Operating System	Windows 7+ / macOS 10.12+ / Linux (any modern distro)	Development and hosting platform
Web Browser	Chrome 90+, Firefox 85+, Edge 90+	Client-side access and testing
Web Server	Apache 2.4+ or Nginx 1.18+	Serving PHP files and static assets
PHP	Version 7.4 or higher	Server-side API and backend logic
MySQL	Version 5.7 or higher	Relational database for persistent storage
phpMyAdmin	Any recent version	Database administration interface (optional)
Text Editor / IDE	Visual Studio Code, Sublime Text, PhpStorm	Code editing and development
XAMPP / WAMP / MAMP	Latest stable release	Bundled development environment
Git	2.x or higher	Version control (recommended)

2.3 Development Environment Setup

The project was developed using XAMPP (Cross-Platform Apache, MySQL, PHP, and Perl) on a local development machine running Windows. XAMPP provides a complete, pre-configured web development stack in a single installer, making it the most convenient environment for PHP/MySQL development on personal computers.

Step-by-Step Setup Procedure:

10. Download and install XAMPP from https://www.apachefriends.org/ for the appropriate operating system.
11. Launch the XAMPP Control Panel and start the Apache and MySQL services.
12. Navigate to the htdocs directory (typically C:\xampp\htdocs on Windows or /Applications/XAMPP/htdocs on macOS).
13. Create a new project folder named sai_deepak within htdocs and place all project files inside it.
14. Open a web browser and navigate to http://localhost/phpmyadmin to access the database management interface.
15. Create a new database named sai_deepak using phpMyAdmin.
16. Import the sai_deepak.sql schema file to create and seed all required tables.
17. Open http://localhost/sai_deepak/index.html in a browser to access the application.
18. To test offline mode, set USE_PHP_API = false in app.js and reload the page.

2.4 Deployment Requirements

For production deployment, the following additional considerations apply beyond basic development environment setup.

The server must have a valid SSL/TLS certificate installed so the application is served over HTTPS. This is essential for protecting user credentials and payment-related data in transit.

- The MySQL database user account used by api.php must have minimum necessary privileges: SELECT, INSERT, UPDATE on the sai_deepak database tables. It should not have DROP or CREATE TABLE privileges in production.
- The api.php file should include CORS (Cross-Origin Resource Sharing) headers configured to allow only the known frontend domain, not wildcard (*) origins, in production.
- PHP error display should be disabled in production (display_errors = Off in php.ini) and errors should be logged to a server log file rather than output to the browser.

- The USE_PHP_API constant in app.js must be set to true and the API_URL must point to the correct server path for full-stack production operation.

3. SYSTEM ARCHITECTURE

This chapter describes the architectural design of the Sai Deepak Film Booking System, including its three-tier structure, data flow model, component interaction pattern, and dual-mode operation strategy.

3.1 Three-Tier Architecture

The system follows the classic three-tier web application architecture pattern, which separates concerns into three distinct layers: the Presentation Layer, the Application (Business Logic) Layer, and the Data Layer. This separation provides clear modularity, easier maintenance, and the ability to modify any layer independently without affecting the others.

Tier 1 — Presentation Layer (Frontend):

The presentation layer consists of the static frontend files served to and rendered by the user's web browser. It includes index.html (the home page), booking.html (the three-step booking wizard), style.css (the complete stylesheet), app.js (core application logic), and booking.js (booking-specific logic). This layer is responsible for rendering the user interface, capturing user input, managing client-side application state, and communicating with the backend via the Fetch API.

Tier 2 — Application Layer (Backend):

The application layer is implemented as a single PHP file: api.php. This file receives all HTTP requests from the frontend, validates input, executes the appropriate business logic (authentication, booking creation, cancellation), performs database queries, and returns structured JSON responses. The choice of a single-file API architecture — sometimes called a 'monolithic API' — simplifies deployment and reduces complexity for an academic project of this scale.

Tier 3 — Data Layer (Database):

The data layer is a MySQL 5.7+ relational database named sai_deepak. It stores all persistent application data across four tables: users, movies, showtimes, and bookings. The database enforces referential integrity through foreign key constraints between the bookings and users tables, and between the showtimes and movies tables.

Tier	Layer Name	Technologies	Responsibility
1	Presentation	HTML5, CSS3, JavaScript ES6+	UI rendering, user input, client state
2	Application	PHP 7.4+, Fetch API	Business logic, authentication, API routing
3	Data	MySQL 5.7+	Persistent storage, relational data management

3.2 Data Flow Diagram

The data flow within the system can be described as a request-response cycle initiated by the user and propagated through each architectural tier. The following description traces the complete data flow for a typical booking action.

Step 1 — User Interaction:

The user interacts with the browser interface — for example, clicking the 'Book Now' button on a movie card in the home page grid. This triggers a JavaScript event listener in app.js or booking.js.

Step 2 — Client-Side State Update:

The JavaScript code updates the application state in memory (for example, storing the selected movie ID in a variable) and manipulates the DOM to reflect the new state (for example, navigating to booking.html with the selected movie pre-populated).

Step 3 — API Request:

When an action requires server-side processing (registration, login, booking submission, booking retrieval, cancellation), the JavaScript code uses the Fetch API to send an asynchronous HTTP request to api.php.

The request includes an 'action' query parameter specifying the operation and, for POST requests, a JSON body containing the required data.

Step 4 — Server Processing:

The api.php file receives the request, reads the action parameter, validates the input, and executes the corresponding database operation using MySQLi prepared statements. It then constructs a JSON response object containing a success flag and the relevant result data (for example, the newly created user object or the generated booking ID).

Step 5 — Response Handling:

The Fetch API promise resolves with the JSON response. The JavaScript callback processes the response, updates the application state accordingly, modifies the DOM to reflect the outcome (for example, displaying the booking success modal), and shows a toast notification to inform the user of the result.

Offline Fallback Flow:

When USE_PHP_API is set to false or when a Fetch request fails (network error or server unavailability), the JavaScript code falls back to reading from and writing to the browser's localStorage API. All operations that would normally involve an API call instead use synchronous localStorage.getItem() and localStorage.setItem() calls, ensuring the application remains functional without any server infrastructure.

3.3 Component Interaction Model

The following table summarises the interaction relationships between the major components of the system.

Source Component	Target Component	Interaction Type	Data Exchanged
index.html	app.js	Script load	DOM event bindings, app initialisation
booking.html	booking.js + app.js	Script load	Booking wizard state, seat map
app.js / booking.js	api.php	HTTP (Fetch API)	JSON request/response pairs
api.php	MySQL (sai_deepak)	MySQLi queries	SQL queries and result sets
app.js	localStorage	Browser API	User session, offline bookings
booking.js	localStorage	Browser API	Fallback booking records

3.4 Dual-Mode Operation Strategy

One of the most architecturally significant design decisions in the project is the implementation of a dual-mode operation strategy. This design pattern allows the application to function fully in two distinct modes:

Mode 1 — Online Full-Stack Mode (USE_PHP_API = true):

In this mode, all data operations are performed through the PHP API and MySQL database. User accounts, movie data, bookings, and cancellations are all stored persistently on the server. This mode is appropriate for production deployment and provides the full multi-user experience where bookings made by one user are visible to the system and can potentially affect seat availability for other users.

Mode 2 — Offline localStorage Mode (USE_PHP_API = false):

In this mode, all data operations use the browser's localStorage API. The movie catalogue is sourced from the static MOVIES array defined in app.js. User accounts and bookings are stored locally in the browser's localStorage object. This mode is entirely server-independent and can be demonstrated from any machine without network access or server configuration. The trade-off is that data does not persist across browsers or devices, and there is no multi-user interaction.

The mode toggle is implemented through a single boolean constant (USE_PHP_API) at the top of app.js. All API wrapper functions (apiGet, apiPost) check this flag before deciding whether to issue a Fetch request or perform a localStorage operation. This clean abstraction means that the higher-level booking workflow code in booking.js is entirely unaware of which mode is active, ensuring zero code duplication between modes.

4. MODULES DESCRIPTION

The Sai Deepak Film Booking System is functionally divided into six major modules, each responsible for a distinct area of application behaviour. This chapter provides an in-depth description of each module, including its sub-functions, the files that implement it, and the specific technical mechanisms it uses.

4.1 User Authentication Module

The User Authentication Module is the gateway through which users establish and maintain their identity within the system. It provides two core sub-functions: user registration and user login. Both sub-functions are accessible through overlay modal dialogs that appear on top of the home page without navigating away from the current content.

4.1.1 User Registration

Registration allows a new user to create an account within the system. The registration modal presents three input fields: Full Name, Email Address, and Password. When the user submits the form, the handleRegister() function in app.js is invoked.

If USE_PHP_API is true, handleRegister() sends a POST request to api.php with the action parameter set to 'register' and the form data serialised as a JSON body. On the server side, api.php first checks whether the submitted email address already exists in the users table using a prepared SELECT query. If a duplicate is found, it returns a JSON error response with the message 'Email already registered.' If the email is unique, it hashes the password using PHP's password_hash() function with the PASSWORD_BCRYPT algorithm (cost factor 12 by default) before inserting the new record into the users table. The API returns the newly created user object (excluding the password hash) as a JSON response.

If USE_PHP_API is false, handleRegister() reads the existing users array from localStorage (stored under the key cvUsers), checks for email uniqueness within that array, and if unique, pushes the new user object directly into the array and saves it back to localStorage. In offline mode, passwords are stored in plain text within localStorage — this is an acknowledged limitation discussed in Chapter 11.

Upon successful registration (both online and offline), the returned user object is serialised and stored in localStorage under the key cvUser, establishing the user session. The navigation bar is updated to display the user's name, and the registration modal is closed.

4.1.2 User Login

Login allows an existing user to authenticate and establish a session. The login modal presents two input fields: Email Address and Password. When submitted, the handleLogin() function in app.js is invoked.

If USE_PHP_API is true, handleLogin() sends a POST request to api.php with action='login'. The PHP script retrieves the user record matching the submitted email using a prepared SELECT query. It then calls password_verify() to compare the submitted password against the stored bcrypt hash. If verification succeeds, the user object is returned in the JSON response. If the email does not exist or the password does not match, a JSON error response is returned with the message 'Invalid credentials.'

If USE_PHP_API is false, handleLogin() searches the cvUsers array in localStorage for a user with a matching email, then performs a plain text comparison of the submitted password against the stored value. While functionally equivalent in demonstration, this approach lacks the security properties of bcrypt hashing.

Upon successful login, the user object is stored in localStorage (cvUser), the navbar is updated, and the modal is dismissed. The user remains logged in across page reloads until they explicitly log out or clear their browser storage.

4.1.3 Session Management

Session state is managed entirely on the client side. On every page load, app.js reads the cvUser key from localStorage. If a user object is found, the application considers the user to be logged in, updates the navbar with the user's name, and enables features that require authentication (such as viewing booking history and completing a booking). There is no server-side session token, cookie, or JWT in the current implementation. This is an acknowledged limitation that is discussed further in Chapter 11.

4.2 Movie Browsing Module

The Movie Browsing Module is responsible for presenting the film catalogue to the user and enabling genre-based filtering. It operates primarily on the home page (index.html) and is implemented in app.js.

4.2.1 Movie Data Source

The movie catalogue can originate from two sources depending on the operation mode. In online mode, the loadMovies() function sends a GET request to api.php with action='movies'. The PHP backend executes a SELECT query on the movies table and returns the full list of active films as a JSON array. In offline mode, the static MOVIES constant array defined at the top of app.js serves as the data source. This array contains eight film objects, each with properties for id, title, genre, rating, duration, price, emoji, and description.

4.2.2 Movie Card Rendering

Once the movie data is available (from either source), the renderMovies() function iterates over the array and dynamically creates an HTML card element for each film. Each card displays: an emoji poster image, the film title in Bebas Neue font, a star rating with numeric value, the film duration, the ticket price per seat, and a genre tag. A 'Book' overlay button appears on mouse hover (desktop) or tap (mobile). Clicking this button navigates to booking.html with the selected film's ID encoded as a URL query parameter (e.g., booking.html?movie=stellar-void).

4.2.3 Genre Filtering

A row of filter buttons at the top of the All Screenings section allows users to filter the visible film cards by genre. The available genres are: All, Action, Drama, Sci-Fi, and Horror. When a genre button is clicked, the filterMovies() function applies a filter to the movies array in memory and calls renderMovies() with the filtered subset. The active filter button is visually distinguished with a gold accent border. Filtering is performed entirely client-side without any additional server requests, making it instantaneous.

4.2.4 Hero Section

The top section of the home page features a full-width hero banner showcasing a single featured film — Stellar Void (a Sci-Fi film). The hero section displays a large film title in Bebas Neue, an extended synopsis paragraph, star rating, duration, genre tag, and a prominent 'Book Now' call-to-action button. The background of the hero section uses a CSS grain overlay texture implemented via an SVG filter to simulate the aesthetic of film grain, contributing to the cinematic visual identity of the application.

4.3 Booking Workflow Module

The Booking Workflow Module is the core functional module of the system. It implements the three-step booking wizard accessible through booking.html and booking.js. This module manages complex multi-step state, interactive UI components, and integration with both the API and localStorage fallback.

4.3.1 Step 1 — Film and Showtime Selection

When booking.html is loaded, JavaScript reads the movie URL parameter and pre-selects the corresponding film in the booking wizard's movie selection grid. If no parameter is present, all available films are displayed for selection. The movie grid in the booking wizard uses the same card layout and rendering logic as the home page grid.

Upon selecting a film, the showtime selection section becomes visible with a smooth CSS transition animation. The date picker renders a horizontal row of date pills representing the current day and the following six days (seven days total). Each pill displays the day abbreviation (Mon, Tue, etc.) and the date number. Clicking a pill selects that date and highlights the pill with a gold accent.

Below the date picker, five time slot pills display the available showtimes: 10:30 AM, 1:15 PM, 4:00 PM, 7:30 PM, and 10:00 PM. Each time slot pill also shows a mock remaining seat count (e.g., '48 seats'). In the current implementation, these seat counts are static display values rather than dynamically queried values from the database. The user must select both a date and a showtime before the Continue button for Step 1 becomes active.

4.3.2 Step 2 — Seat Selection

The seat selection interface renders an interactive seat map using a CSS Grid layout. The map consists of seven rows labelled A through G and ten seat positions per row, yielding 70 seats in total. Each seat is rendered as a small square button element with a CSS class reflecting its current state: available, selected, or taken.

The TAKEN constant in booking.js defines a fixed set of seat identifiers (e.g., ['A1', 'A2', 'B5', 'C3', ...]) that are rendered as unavailable (greyed out and non-clickable) at all times. This static definition is an architectural limitation — in a production system, the set of taken seats would be dynamically loaded from the database based on the selected film, date, and showtime.

Clicking an available seat toggles its state between available and selected, updating the CSS class and colour accordingly. The selected seat identifiers are maintained in an in-memory array and are displayed below the seat map in a text summary (e.g., 'Selected: A3, B7, C4'). If the user attempts to select a ninth seat after already selecting eight, a toast notification appears with the message 'Maximum 8 seats per booking' and the click is ignored. The Continue button for Step 2 is enabled only when at least one seat is selected.

4.3.3 Step 3 — Confirmation and Payment

Step 3 presents an order summary card alongside a payment capture form. The order summary displays all booking details collected in the preceding steps: the selected film title and genre, the chosen date and showtime, the list of selected seat identifiers, the per-seat price, and the total amount payable (calculated as seat count multiplied by the per-seat price).

The payment form includes four fields: Full Name, Email Address, Card Number (a 16-digit demonstration field with automatic hyphen formatting every four digits), Card Expiry (MM/YY format), and CVV (3 digits). Each field has client-side validation applied on submission: name and email are checked for non-empty values and basic format, the card number is checked for 16 digits, expiry is checked for MM/YY format, and CVV is checked for exactly 3 numeric digits.

When the user submits the form, submitBooking() in booking.js gathers the full booking payload and calls apiPost('booking', payload). The payload includes: movie_id, movie_title, date, showtime, seats (as a comma-separated string), name, email, and total. The PHP API inserts this into the bookings table and returns the generated booking_id. A success modal then appears, displaying a formatted booking summary with the booking reference number. If the API call fails (network error or server unavailability), the payload is saved to localStorage under cvBookings as a fallback.

4.4 My Bookings Module

The My Bookings Module enables authenticated users to view their complete booking history and cancel active bookings. This module is implemented in app.js and is accessible from the home page after login.

4.4.1 Booking Retrieval

When a logged-in user accesses the My Bookings section, the loadUserBookings() function is invoked. In online mode, it sends a GET request to api.php with action='bookings' and the user's email address as a query parameter. The PHP backend executes a SELECT query on the bookings table filtered by the email column and returns all matching records sorted by creation date in descending order. In offline mode, it reads the cvBookings array from localStorage and filters it by the email address stored in the active user session.

4.4.2 Booking Card Rendering

Each retrieved booking is rendered as a card in the My Bookings section. Each card displays: the film title and genre tag, the booking date and showtime, the full list of seat identifiers, the total amount paid, the booking status badge (green 'Confirmed' or red 'Cancelled'), and — for active confirmed bookings — a Cancel button.

4.4.3 Booking Cancellation

When a user clicks the Cancel button on an active booking card, a confirmation prompt appears asking the user to confirm the cancellation action. Upon confirmation, the cancelBooking(id) function sends a POST request to api.php with action='cancel' and the booking ID. The PHP backend executes an UPDATE query to set the booking status to 'cancelled' for that record. Upon receiving a success response, the booking list is re-fetched and re-rendered to reflect the updated status. In offline mode, the cvBookings array in localStorage is updated directly.

4.5 PHP Backend API Module

The PHP Backend API Module is implemented entirely within the single file api.php. It serves as the sole server-side component of the system and exposes a simple RESTful interface accepting requests via query parameters and returning JSON responses.

4.5.1 Request Routing

The api.php file begins by setting the Content-Type header to application/json and the Access-Control-Allow-Origin header to permit cross-origin requests (appropriate for development; should be restricted in production). It then reads the 'action' query parameter from the request URL to determine which operation to perform. A switch statement routes control to the appropriate handler function based on this action value.

4.5.2 Database Connection

The database connection is established using the MySQLi extension with the mysqli() constructor. Connection parameters (hostname, username, password, database name) are defined as constants at the top of api.php. If the connection fails, the script immediately returns a JSON error response with the message 'Database connection failed' and exits. Prepared statements are used for all database operations to prevent SQL injection attacks.

4.5.3 Supported API Actions

Action	HTTP Method	Input Parameters	Response Data
register	POST	name, email, password	User object (id, name, email, created_at)
login	POST	email, password	User object (id, name, email, created_at)
movies	GET	None	Array of movie objects
booking	POST	movie_id, movie_title, date, showtime, seats, name, email, total	Booking object with booking_id
bookings	GET	email	Array of booking objects for that email
cancel	POST	booking_id	Success confirmation object

4.5.4 Error Handling

Each action handler in api.php wraps its database operations in error checks. If a MySQLi query fails, the error message is captured and returned as a JSON error response. Input validation is performed before executing any query — for example, the register handler checks that name, email, and password are all non-empty before attempting the duplicate email check. All responses use a consistent JSON envelope with a 'success' boolean field and either a 'data' field (on success) or a 'message' field (on error).

4.6 Notification Module

The Notification Module provides real-time feedback to users through a lightweight toast notification system. Toast messages appear as small rectangular cards in the bottom-right corner of the browser viewport, overlaying the main content without interrupting the user's workflow.

The showToast(message, type) function in app.js creates a new div element with the appropriate CSS class (toast-success or toast-error), sets its text content to the provided message, appends it to the toast container element in the DOM, and then uses a CSS transition to animate it into view. After 3,500 milliseconds, the toast is animated out and

removed from the DOM. Multiple toasts can appear simultaneously — each successive toast appears above the previous one in a stacked arrangement.

Toast notifications are triggered throughout the application for the following events: successful registration, login, and logout; booking confirmation; booking cancellation; validation errors in forms (registration, login, payment); seat selection limit exceeded; and API communication errors.

5. DATABASE DESIGN

This chapter presents the complete database design for the Sai Deepak Film Booking System, including the entity relationship model, individual table schemas, SQL DDL statements, and a discussion of normalisation principles applied.

5.1 Entity Relationship Overview

The database sai_deepak consists of four tables: users, movies, showtimes, and bookings. The relationships between these tables are as follows:

- A user can have zero or more bookings (one-to-many relationship: users → bookings).
- A movie can have zero or more showtimes (one-to-many relationship: movies → showtimes).
- A booking references one movie and optionally one user (the user_id is nullable to support guest bookings).

The entity relationship structure can be summarised in the following key relationship table:

Parent Entity	Child Entity	Relationship Type	Foreign Key
users	bookings	One-to-Many (optional)	bookings.user_id → users.id
movies	showtimes	One-to-Many	showtimes.movie_id → movies.id
movies	bookings	One-to-Many (via movie_id)	bookings.movie_id → movies.id (string)

5.2 Table Descriptions

5.2.1 users Table

The users table stores the identity and authentication credentials of all registered users.

Field	Type	Constraints	Description
id	INT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Unique user identifier
name	VARCHAR(120)	NOT NULL	User's full display name
email	VARCHAR(180)	NOT NULL, UNIQUE	Email address used for login
password	VARCHAR(255)	NOT NULL	bcrypt-hashed password (60+ characters)
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Account creation timestamp

The email column is defined with a UNIQUE constraint to prevent duplicate registrations with the same address. The password column stores bcrypt hashes, which are always 60 characters long for cost factor 10 (up to 72 characters for higher cost factors), making VARCHAR(255) a safe choice for future algorithm changes.

5.2.2 movies Table

The movies table stores the full catalogue of films available for booking.

Field	Type	Constraints	Description
id	VARCHAR(80)	PRIMARY KEY	Slug-style unique identifier (e.g., 'stellar-void')
title	VARCHAR(150)	NOT NULL	Full display title of the film
genre	VARCHAR(50)	NOT NULL	Genre category (Action, Drama, Sci-Fi, Horror)
rating	DECIMAL(3,1)	NULL	Star rating out of 10.0
duration	VARCHAR(30)	NULL	Display string (e.g., '2h 28m')
price	DECIMAL(6,2)	NULL	Per-seat ticket price in Indian Rupees
emoji	VARCHAR(10)	NULL	Display emoji for the movie card
description	TEXT	NULL	Short synopsis or tagline
active	TINYINT(1)	DEFAULT 1	Soft delete flag (1=active, 0=hidden)

The use of a VARCHAR primary key (the slug) rather than an auto-incremented integer is a deliberate design choice. Slug-based IDs (e.g., 'stellar-void') are human-readable and can be used directly in URL query parameters without any additional lookup. The active flag enables soft deletion, preserving historical booking records even if a film is removed from the catalogue.

5.2.3 showtimes Table

The showtimes table stores the time slots available for each film. In the current implementation, all films share the same set of five showtimes, but the table structure supports per-film showtime configurations.

Field	Type	Constraints	Description
id	INT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Unique showtime record identifier
movie_id	VARCHAR(80)	FOREIGN KEY → movies.id	The film this showtime applies to
show_time	VARCHAR(20)	NOT NULL	Time string (e.g., '10:30 AM', '7:30 PM')

5.2.4 bookings Table

The bookings table is the central transactional table of the system. It stores a complete record of every booking made, including denormalised copies of the film title and seat data for display efficiency.

Field	Type	Constraints	Description
id	INT UNSIGNED	PRIMARY KEY, AUTO_INCREMENT	Unique booking reference number
user_id	INT UNSIGNED	NULLABLE FK → users.id	Linked user (null for guest bookings)
movie_id	VARCHAR(80)	NOT NULL	References movies.id slug
movie_title	VARCHAR(150)	NOT NULL	Denormalised title for fast display
date	DATE	NOT NULL	Selected viewing date

Field	Type	Constraints	Description
showtime	VARCHAR(20)	NOT NULL	Selected showtime string
seats	TEXT	NOT NULL	Comma-separated seat identifiers (e.g., 'A3,B7,C4')
name	VARCHAR(150)	NOT NULL	Booker's full name
email	VARCHAR(180)	NOT NULL	Booker's email address
total	DECIMAL(8,2)	NOT NULL	Total payment amount in Rupees
status	ENUM	('confirmed','cancelled')	Current booking status
created_at	DATETIME	DEFAULT CURRENT_TIMESTAMP	Booking creation timestamp

The seats field uses a TEXT column storing a comma-separated list of seat identifiers rather than a separate many-to-many junction table. While this denormalised approach is less flexible than a normalised design, it is appropriate for the scale of this project and avoids complex JOIN queries when displaying booking details.

5.3 SQL Schema (DDL Statements)

The complete database schema is defined in the sai_deepak.sql file. The key DDL statements for table creation are as follows:

Create users table:

CREATE TABLE users (id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY, name VARCHAR(120) NOT NULL, email VARCHAR(180) NOT NULL UNIQUE, password VARCHAR(255) NOT NULL, created_at DATETIME DEFAULT CURRENT_TIMESTAMP) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

Create movies table:

CREATE TABLE movies (id VARCHAR(80) PRIMARY KEY, title VARCHAR(150) NOT NULL, genre VARCHAR(50) NOT NULL, rating DECIMAL(3,1), duration VARCHAR(30),

price DECIMAL(6,2), emoji VARCHAR(10), description TEXT, active TINYINT(1) DEFAULT 1) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

Create bookings table:

CREATE TABLE bookings (id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY, user_id INT UNSIGNED, movie_id VARCHAR(80) NOT NULL, movie_title VARCHAR(150) NOT NULL, date DATE NOT NULL, showtime VARCHAR(20) NOT NULL, seats TEXT NOT NULL, name VARCHAR(150) NOT NULL, email VARCHAR(180) NOT NULL, total DECIMAL(8,2) NOT NULL, status ENUM('confirmed','cancelled') DEFAULT 'confirmed', created_at DATETIME DEFAULT CURRENT_TIMESTAMP, FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

5.4 Database Normalization

The database design broadly adheres to Third Normal Form (3NF). Each table has a single-column primary key with no repeating groups (1NF). All non-key columns in each table are fully functionally dependent on the entire primary key (2NF). There are no transitive dependencies where a non-key column depends on another non-key column (3NF), with the deliberate exception of the denormalised movie_title in the bookings table.

The intentional denormalisation of movie_title in the bookings table is a pragmatic design decision. Since movie titles can theoretically be updated in the movies table, storing a snapshot of the title at the time of booking ensures that historical booking records always display the correct title even if the movie record is later modified or soft-deleted. This pattern is commonly employed in e-commerce order systems for product names and prices.

6. FILE STRUCTURE

This chapter documents the file structure of the Sai Deepak Film Booking System, explaining the purpose and role of each file within the project directory.

6.1 File Organization

The project follows a flat file structure: all files reside in a single directory without subdirectories. This approach prioritises simplicity of deployment, particularly in shared hosting environments where complex directory configurations may not be supported. The complete file list is as follows:

File Name	Type	Size (Approximate)	Description
index.html	HTML	~4 KB	Main landing page with hero section and movie grid
booking.html	HTML	~5 KB	Three-step booking wizard page
style.css	CSS	~12 KB	Complete dark-theme stylesheet with animations
app.js	JavaScript	~8 KB	Core application logic: data, auth, API, toasts
booking.js	JavaScript	~10 KB	Booking wizard: seat map, steps, submission
api.php	PHP	~6 KB	REST API for auth, movies, bookings, cancellations
sai_deepak.sql	SQL	~3 KB	Database schema and seed data
README.md	Markdown	~2 KB	Developer setup and configuration documentation

6.2 File Roles and Responsibilities

6.2.1 index.html — Home Page

index.html defines the structural markup of the application's home page. It contains the HTML skeleton for the navigation bar, the hero section banner, the genre filter button row, the movie grid container (populated dynamically by JavaScript), the My Bookings section (visible only when authenticated), and the login/registration modal overlays. It loads

style.css for styling and app.js for behaviour. The HTML is minimal — most content is generated dynamically by JavaScript rather than being hardcoded in the markup.

6.2.2 booking.html — Booking Wizard Page

booking.html defines the structural markup of the three-step booking wizard. It contains the step indicator bar (with step numbers 01, 02, 03 and connecting lines), the three step content panels (initially hidden except Step 1), the seat map container, the order summary card, and the payment form. It loads both app.js and booking.js. The step navigation logic, seat map rendering, and form validation are all handled by booking.js.

6.2.3 style.css — Stylesheet

style.css is the complete stylesheet for the entire application. It defines CSS custom properties (variables) for the colour palette (dark backgrounds, gold accents, charcoal greys), the Google Fonts imports for Bebas Neue and Barlow, the global reset rules, the navigation bar layout, the hero section with grain overlay, the movie card grid and hover effects, the booking wizard step indicator, the seat map grid and seat state colours, the modal overlay styles, the toast notification animation keyframes, and the responsive media query breakpoints. All visual effects including hover transitions, step completion animations, and toast slide-in/slide-out are implemented entirely in CSS.

6.2.4 app.js — Core Application Logic

app.js is the primary JavaScript file loaded by both index.html and booking.html. It contains: the USE_PHP_API configuration constant and API_URL base path; the static MOVIES data array (used in offline mode); the apiGet() and apiPost() wrapper functions implementing the dual-mode switching logic; the handleRegister() and handleLogin() authentication functions; the loadMovies() and renderMovies() functions for the movie grid; the filterMovies() function for genre filtering; the loadUserBookings() and renderBookings() functions for the My Bookings section; the cancelBooking() function; the modal open/close functions; and the showToast() function for notifications.

6.2.5 booking.js — Booking Wizard Logic

booking.js is loaded exclusively by booking.html. It handles all booking-wizard-specific behaviour: reading the movie URL parameter and pre-selecting the appropriate film; rendering the date picker pills for the next seven days; rendering the time slot pills; managing the currently selected film, date, and showtime in memory; rendering the interactive seat map grid; handling seat click events and enforcing the eight-seat limit; updating the order summary card; validating the payment form; and calling submitBooking() to send the booking payload to the API (or localStorage) upon confirmation.

6.2.6 api.php — REST API

api.php is the sole server-side file. It establishes the MySQL database connection, reads the HTTP request method and action parameter, and routes to one of six action handlers (register, login, movies, booking, bookings, cancel). Each handler uses MySQLi prepared statements to safely interact with the database and returns a structured JSON response. The file is stateless — each HTTP request is processed independently without maintaining any server-side session state between requests.

6.2.7 sai_deepak.sql — Database Schema

sai_deepak.sql contains the complete MySQL DDL (Data Definition Language) statements to create the four database tables (users, movies, showtimes, bookings) and the DML (Data Manipulation Language) INSERT statements to populate the movies and showtimes tables with seed data for the eight films and their five showtimes each.

7. USER INTERFACE DESIGN

This chapter describes the user interface design of the Sai Deepak Film Booking System in detail, covering the design philosophy, individual page layouts, interactive components, modal dialogs, toast notifications, and responsive design approach.

7.1 Design Philosophy

The user interface adopts a cinematic dark-theme aesthetic inspired by the visual language of premium movie ticketing platforms such as BookMyShow and IMAX Online. The design philosophy is rooted in three core principles: immersion, clarity, and elegance.

Immersion:

The colour palette is built around deep blacks (#0a0a0a background), charcoal greys (#1a1a1a, #2a2a2a), and gold accent colours (#c9a84c, #e8c547). This palette evokes the atmosphere of a darkened cinema auditorium illuminated by the warm glow of the screen. The hero section uses a CSS SVG grain filter overlay to simulate the texture of analogue film grain, reinforcing the cinematic identity. The Bebas Neue display typeface — a condensed, all-caps letterform widely used in film poster design — is applied to all major headings, titles, and labels.

Clarity:

Despite the atmospheric visual treatment, functional clarity is maintained throughout. Interactive elements such as buttons, seat cells, date pills, and time slot pills are clearly distinguishable from decorative content through consistent use of borders, colour contrast, and hover/active state transitions. The booking wizard's three-step structure with numbered step indicators (01, 02, 03) provides a clear mental model of the user's progress through the reservation process. Disabled states (e.g., the Continue button before selections are complete) use reduced opacity to communicate unavailability without confusing the user.

Elegance:

Animation and transition effects are applied with restraint. CSS transitions on hover states, modal fade-ins, toast slide-ins, and step completion animations create a polished, professional feel without overwhelming the interface with motion. Typography is set with generous line spacing and the Barlow typeface — a grotesque sans-serif with excellent legibility — for all body text, creating an optimal reading experience even on smaller screens.

7.2 Home Page (index.html)

The home page is structured into four distinct vertical sections arranged in a single-column flow:

Navigation Bar:

A sticky navigation bar at the top of the page contains the brand logo ('SAI DEEPAK CINEMAS' in Bebas Neue), navigation links to the main sections of the home page, and a user authentication control in the top-right corner. When no user is logged in, this control displays a 'Sign In' button. When a user is authenticated, it displays the user's name and a 'Logout' link.

Hero Section:

The hero section occupies the full viewport height on desktop screens. It features a large textured background, the featured film title in oversized Bebas Neue lettering, a short synopsis, film metadata (rating badge, duration chip, genre tag), and a gold 'Book Now' call-to-action button. The grain overlay texture is applied as a pseudo-element using a CSS background with an SVG data URI filter effect.

Genre Filter Row:

A horizontal row of pill-shaped filter buttons (All, Action, Drama, Sci-Fi, Horror) allows users to filter the movie grid. The active filter is indicated by a gold border and slightly lighter background. On narrow screens, this row becomes horizontally scrollable.

Movie Grid (All Screenings):

The movie catalogue is displayed in a responsive CSS Grid layout. On wide desktop screens (1200px+), the grid displays four cards per row. On medium screens (768–1199px), three cards per row. On narrow screens (below 768px), two cards per row. On very narrow screens (below 480px), a single-column layout. Each card has a slight elevation shadow and a hover effect that scales the card slightly and reveals the 'Book' overlay button.

7.3 Booking Page (booking.html)

The booking page is structured around a central three-step wizard with a visual progress indicator at the top of the content area.

Step Indicator:

Three numbered step circles (01, 02, 03) are connected by horizontal lines. The active step's circle is filled with the gold accent colour. Completed steps display a checkmark icon. Steps that have not yet been reached appear in a muted grey. The indicator updates dynamically as the user progresses through the wizard.

Step 1 — Film & Showtime Selection:

The movie grid is displayed identically to the home page grid (same card layout and hover effects). Below the grid, the showtime panel slides into view when a film is selected. The date picker renders as a horizontal scrollable row of date pills, each showing the day abbreviation and date number. Below the date row, the five time slot pills display the available showtimes with seat count indicators.

Step 2 — Seat Map:

The seat map is rendered in a centred container with a 'SCREEN' label at the top to orient the user. The CSS Grid layout creates uniform seat cells with small gaps between them. A legend below the map explains the three seat states: a gold square for Available, a grey square for Taken, and a bright accent square for Selected. The seat identifiers (A3, B7, etc.) are displayed in a summary line below the legend.

Step 3 — Confirmation:

The confirmation step uses a two-column layout on desktop: the left column shows the order summary card (film details, date, seats, total), and the right column shows the payment form. On mobile, these stack vertically with the order summary on top. The payment form fields are styled with dark backgrounds and gold focus borders, consistent with the overall dark theme.

7.4 Modals

Login and registration are handled through CSS overlay modals. The modal backdrop is a semi-transparent dark overlay (rgba(0,0,0,0.8)) that covers the entire viewport, effectively dimming the background content and focusing the user's attention on the modal form. The modal card itself is centred both horizontally and vertically using CSS flexbox. It animates into view with a CSS transform-based scale-up transition (from 0.8 to 1.0 scale with an opacity fade from 0 to 1) over 250ms.

A close button (×) in the top-right corner of the modal dismisses it without navigating away. Clicking the backdrop also dismisses the modal.Users can switch between the login and registration modals by clicking the 'Register here' or 'Already have an account? Login' links within each modal. This switching is handled by hiding one modal and showing the other without any animation interruption.

7.5 Toast Notifications

The toast notification component provides non-blocking, contextual feedback messages to the user. Toast messages appear in the bottom-right corner of the viewport, sliding in from the right via a CSS translateX animation. Each toast persists for 3.5 seconds before sliding back out and being removed from the DOM. Success toasts use a green left-border accent; error toasts use a red left-border accent. The toast text is set in Barlow at 14px with sufficient contrast against the dark background.

Multiple toasts can coexist simultaneously. Each new toast is appended below the existing ones in the stack. The toast container uses a high z-index value to ensure toasts appear above all other UI elements including modal backdrops.

7.6 Responsive Design

The application uses a mobile-first responsive design approach implemented through CSS media query breakpoints. The primary breakpoints are at 480px (small mobile), 768px (tablet/large mobile), and 1024px (desktop). The navigation bar collapses its link list on narrow screens. The movie grid transitions from four columns to two columns to one column as screen width decreases. The booking wizard step indicator condenses on narrow screens by abbreviating step labels. The seat map scales to fit within the viewport on mobile through a combination of reduced cell sizes and horizontal scroll enablement if necessary. The two-column confirmation layout collapses to a single column on mobile.

8. FUNCTIONAL DESCRIPTION

This chapter provides a detailed functional description of the system's key operational workflows, describing the exact sequence of actions, code functions, and data transformations involved in each user-initiated scenario.

8.1 User Registration and Login

The registration and login workflows are the entry points for all personalised features of the system. When a visitor clicks 'Sign In' in the navigation bar, the openLoginModal() function is called, which sets the modal's display property to 'flex' and triggers the CSS scale-up animation.

For registration, the user fills in the three-field form and clicks 'Register'. The handleRegister(event) function is invoked. It first calls event.preventDefault() to prevent the default HTML form submission behaviour. It reads the values from the name, email, and password input elements. It then calls apiPost('register', { name, email, password }). The apiPost function checks USE_PHP_API. If true, it uses fetch(API_URL + '?action=register', { method: 'POST', headers: { Content-Type: application/json }, body: JSON.stringify(payload) }) and awaits the JSON response. If false, it performs the equivalent localStorage operation. On success, it calls storeUser(data.user), which calls localStorage.setItem('cvUser', JSON.stringify(user)), then calls updateNavbar() to display the user's name, calls closeModal(), and calls showToast('Registration successful! Welcome, ' + user.name, 'success').

The login flow follows an identical pattern except the API action is 'login' and the payload contains only email and password. The PHP backend's login handler executes a SELECT query for the email, then calls password_verify(submittedPassword, storedHash). If this returns true, the user object is returned; otherwise, a JSON error with 'Invalid credentials' is returned.

8.2 Booking Submission

The booking submission workflow is the most complex operational sequence in the application. It spans both the booking wizard UI in booking.js and the API handler in api.php. The following is the complete step-by-step functional description.

When the user completes Step 3 payment form validation and clicks 'Confirm Booking', the handlePaymentSubmit(event) function is invoked. It validates all four payment fields using regular expressions: card number must match /^\d{16}$/, expiry must match /^(0[1-9]|1[0-2])\/\d{2}$/, and CVV must match /^\d{3}$/. If any field fails validation, an appropriate toast is shown and the function returns early. If all fields pass, the booking payload object is assembled: { movie_id: selectedMovie.id, movie_title: selectedMovie.title, date: selectedDate, showtime: selectedShowtime, seats: selectedSeats.join(','), name: paymentName.value, email: paymentEmail.value, total: totalAmount }. The function then calls submitBooking(payload).

Inside submitBooking(), apiPost('booking', payload) is called. In online mode, the Fetch request is sent to api.php?action=booking. The PHP handler reads the JSON body, inserts a new record into the bookings table using a prepared INSERT statement, and returns the generated auto-increment ID as booking_id. Back in the browser, the success modal is displayed with the booking reference number and full summary. In offline mode, the payload is given a locally generated ID (Date.now()), a 'confirmed' status, and a current timestamp, then pushed into the cvBookings array in localStorage.

8.3 Booking Cancellation

The cancellation workflow is initiated when a user clicks the 'Cancel' button on one of their booking history cards in the My Bookings section. The cancelBooking(bookingId) function is called with the numeric ID of the target booking. A window.confirm() dialog prompts the user to confirm: 'Are you sure you want to cancel this booking?' If the user clicks Cancel in the dialog, the function returns early. If they click OK, apiPost('cancel', { booking_id: bookingId }) is called.

In online mode, the PHP handler executes: UPDATE bookings SET status = 'cancelled' WHERE id = ? using a prepared statement with the booking ID. On success, it returns {

success: true }. Back in the browser, a success toast is shown ('Booking cancelled successfully.') and loadUserBookings() is called to re-fetch and re-render the bookings list, which now shows the updated 'Cancelled' status badge for the affected booking.

8.4 API Toggle and Graceful Degradation

The dual-mode operation is controlled by the USE_PHP_API constant at the top of app.js. Setting this to true enables full-stack PHP/MySQL operation. Setting it to false activates the localStorage fallback mode. The apiGet() and apiPost() wrapper functions both contain a conditional check: if (!USE_PHP_API) { return performLocalStorageOperation(action, params); }.

The localStorage fallback implementations mirror the structure of the PHP API responses — they return the same JSON-like object structure with { success: true, data: result } envelopes — ensuring that the calling code in app.js and booking.js never needs to distinguish between the two modes. This architectural decision significantly reduces code complexity and makes the mode switching fully transparent to the higher-level workflow functions.

In the event that USE_PHP_API is true but an individual Fetch request fails (e.g., due to a network timeout or server error), the apiPost() function catches the rejection and falls back to the localStorage operation for that specific action. This ensures that even in mixed-connectivity environments, the booking workflow completes successfully and data is preserved locally.

8.5 Genre Filtering Workflow

The genre filtering workflow is entirely client-side and requires no server communication. When the movie data has been loaded (from either source), it is stored in the module-scoped allMovies array in app.js. When the user clicks a genre filter button, the filterMovies(genre) function is called with the selected genre string. If the genre is 'All', the function calls renderMovies(allMovies) with the complete array. Otherwise, it calls renderMovies(allMovies.filter(m => m.genre === genre)). The active filter button's CSS class is updated to visually indicate the current selection.

9. TECHNOLOGY STACK

This chapter provides a detailed technical overview of each technology used in the Sai Deepak Film Booking System, including justification for the selection of each technology and notes on specific features utilised.

Layer	Technology	Version	Purpose
Markup	HTML5	Living Standard	Page structure and semantic content
Styling	CSS3	Latest W3C	Dark-theme UI, animations, responsive layout
Client Logic	Vanilla JavaScript	ES6+ (ES2015+)	DOM manipulation, API calls, state management
Server Logic	PHP	7.4+	REST API, authentication, database interaction
Database	MySQL	5.7+	Persistent relational data storage
Typography	Google Fonts	CDN	Bebas Neue (display), Barlow (body)
HTTP Client	Fetch API	Browser Native	Asynchronous API communication
Storage Fallback	localStorage	Browser Native	Offline data persistence
DB Extension	MySQLi	PHP Built-in	Prepared statements and safe DB queries

9.1 HTML5

HTML5 provides the structural foundation of both pages. Semantic HTML5 elements including header, nav, main, section, article, and footer are used throughout to improve accessibility and search engine discoverability. The data-* attribute mechanism is used to store film IDs and other metadata on HTML elements for retrieval by JavaScript event handlers without requiring global variables. The HTML5 History API is not used in this project as all navigation is handled through URL parameters and direct link navigation.

9.2 CSS3

CSS3 provides all visual styling and animation. Key CSS3 features utilised include: CSS Custom Properties (variables) for the colour palette, enabling consistent theming

throughout; CSS Grid for the movie card grid and seat map layouts; CSS Flexbox for navigation bar, modal centering, and step indicator layout; CSS Transitions and Animations for hover effects, modal appearances, and toast notifications; CSS @keyframes for the grain overlay animation; and CSS Media Queries for responsive breakpoints. The decision to use pure CSS without a preprocessor (Sass/Less) or a utility framework (Tailwind/Bootstrap) keeps the build process simple and the file self-contained.

9.3 Vanilla JavaScript (ES6+)

The choice of Vanilla JavaScript (standard browser JavaScript without frameworks or libraries) is a deliberate pedagogical decision. It forces engagement with core language features — arrow functions, template literals, destructuring assignment, spread operators, array methods (map, filter, reduce, forEach), async/await, Promises, and the Fetch API — rather than allowing a framework to abstract these away. The ES6 module system is not used; instead, app.js and booking.js operate as classic browser scripts with shared global scope, which simplifies the file loading setup.

9.4 PHP 7.4+

PHP serves as the server-side language for the REST API. PHP 7.4 is specified as the minimum version because it introduced property type declarations and numeric literal separators, and it is the oldest version still receiving security patches at the time of development. Key PHP features used include: the MySQLi extension with prepared statements for safe database interaction; password_hash() with PASSWORD_BCRYPT for secure password storage; password_verify() for authentication; json_encode() and json_decode() for JSON serialisation; and $_SERVER['REQUEST_METHOD'] for distinguishing GET and POST requests.

9.5 MySQL 5.7+

MySQL 5.7 is specified as the minimum because it introduced native JSON column support (though not used in this project), improved InnoDB performance, and is the standard version available in XAMPP packages. The InnoDB storage engine is used for all tables because it supports foreign key constraints and ACID-compliant transactions,

which are important for data integrity in a booking system. The utf8mb4 character set is specified for all tables to ensure full Unicode support including emoji characters used in the movies table.

9.6 Decision Against Frameworks

The project deliberately avoids frontend frameworks such as React, Vue.js, or Angular, and backend frameworks such as Laravel, Symfony, or CodeIgniter. This decision was made for several reasons. Firstly, the project is primarily an academic exercise aimed at demonstrating understanding of core web technologies rather than framework proficiency. Secondly, avoiding frameworks eliminates build tools (webpack, Babel, npm), reducing setup complexity and enabling the frontend to be tested by simply opening an HTML file in a browser. Thirdly, the project's relatively modest scope — a few pages, a handful of API endpoints — does not justify the overhead of a full framework. The result is a system that any developer can read, understand, and modify without knowledge of any specific framework's conventions.

10. TESTING

This chapter describes the testing approach employed for the Sai Deepak Film Booking System, presents the complete set of test cases with results, and notes browser compatibility findings.

10.1 Testing Approach

The system was tested through manual functional testing. Given the academic nature of the project and the absence of an automated testing framework, all tests were performed by directly interacting with the application in a web browser and verifying the outcome against the expected result. Testing was conducted in two modes:

- Online Mode: Testing with the PHP backend and MySQL database active (XAMPP running, USE_PHP_API = true). This tests the full-stack integration including database persistence, bcrypt password handling, and API response handling.
- Offline Mode: Testing with USE_PHP_API = false. This tests the localStorage fallback path for all data operations and verifies that the application remains fully functional without any server infrastructure.

Module-level testing was performed first for each major module (authentication, movie browsing, booking wizard, booking management) before conducting end-to-end workflow tests covering the complete user journey from registration through booking confirmation to cancellation.

10.2 Test Cases

Test ID	Module	Test Case	Expected Result	Actual Result	Status
TC-01	Auth	Register with valid name, email, password	Account created, user logged in, success toast shown, name appears in navbar	Matched expected	Pass
TC-02	Auth	Register with already-used email	Error toast: 'Email already registered', form stays open	Matched expected	Pass
TC-03	Auth	Register with empty name field	Validation error, form not submitted	Matched expected	Pass
TC-04	Auth	Login with valid credentials	User authenticated, name in navbar, modal closes	Matched expected	Pass
TC-05	Auth	Login with incorrect password	Error toast: 'Invalid credentials', modal stays open	Matched expected	Pass
TC-06	Auth	Login with unregistered email	Error toast: 'Invalid credentials'	Matched expected	Pass
TC-07	Movies	Browse home page movie grid	All 8 movies displayed with correct title, genre, rating, duration, price	Matched expected	Pass
TC-08	Movies	Filter movies by 'Action' genre	Only Action films displayed;	Matched expected	Pass

Test ID	Module	Test Case	Expected Result	Actual Result	Status
			other genres hidden		
TC-09	Movies	Filter by 'Sci-Fi' then switch to 'All'	Sci-Fi films shown, then all 8 restored on 'All' click	Matched expected	Pass
TC-10	Booking	Select a movie on booking page	Showtime section appears below movie grid	Matched expected	Pass
TC-11	Booking	Select a date from date picker	Date pill highlighted in gold	Matched expected	Pass
TC-12	Booking	Select a time slot	Time slot pill highlighted, Continue button enabled	Matched expected	Pass
TC-13	Booking	Attempt to continue without selecting time	Continue button remains disabled	Matched expected	Pass
TC-14	Booking	Select 5 seats on seat map	5 seats highlighted, seat IDs shown below map	Matched expected	Pass
TC-15	Booking	Attempt to select 9th seat	Toast: 'Maximum 8 seats per booking', 9th selection ignored	Matched expected	Pass
TC-16	Booking	Click taken seat	No response, seat remains greyed out	Matched expected	Pass

Test ID	Module	Test Case	Expected Result	Actual Result	Status
TC-17	Booking	Complete payment form with valid data	Booking submitted, success modal shows with booking ID and summary	Matched expected	Pass
TC-18	Booking	Submit payment with invalid card number	Validation error toast, form not submitted	Matched expected	Pass
TC-19	Booking	Submit booking while not logged in	Login modal opens, booking paused	Matched expected	Pass
TC-20	Bookings	View My Bookings after confirmed booking	Booking card shows film, date, seats, total, 'Confirmed' badge	Matched expected	Pass
TC-21	Bookings	Cancel an active booking	Confirm dialog shown; on confirm: status changes to 'Cancelled' in list	Matched expected	Pass
TC-22	Offline	Complete full booking flow with USE_PHP_API=false	Booking saved to localStorage, success modal shown	Matched expected	Pass
TC-23	Offline	View bookings saved in offline mode	Offline bookings displayed correctly in My Bookings section	Matched expected	Pass

Test ID	Module	Test Case	Expected Result	Actual Result	Status
TC-24	Offline	Register and login in offline mode	Account stored in localStorage, login verified client-side	Matched expected	Pass
TC-25	UI	Resize browser window to mobile width	Movie grid collapses to 1 column, booking wizard adapts, seat map scrollable	Matched expected	Pass

10.3 Browser Compatibility

The application was tested on the following browser versions. All features — including the Fetch API, localStorage, CSS Grid, CSS Custom Properties, CSS Animations, and ES6+ JavaScript syntax — functioned correctly across all tested browsers without the need for any polyfills.

Browser	Version Tested	Engine	Result
Google Chrome	120+	Blink / V8	All features pass
Mozilla Firefox	115+	Gecko / SpiderMonkey	All features pass
Microsoft Edge	120+	Blink / V8	All features pass
Safari (macOS)	17+	WebKit / JavaScriptCore	All features pass

11. LIMITATIONS, FUTURE ENHANCEMENTS AND UI SCREEN SHORTS

This chapter presents an honest assessment of the current limitations of the Sai Deepak Film Booking System and outlines a structured roadmap of future enhancements that would be required to elevate the system to production readiness.

11.1 Current Limitations

11.1.1 Demonstration Payment Module

The payment capture form in Step 3 of the booking wizard is a demonstration-only interface. It collects card number, expiry date, and CVV fields and performs basic format validation, but it does not connect to any real payment gateway or financial processing service. No money changes hands, no card verification is performed, and no payment receipt is generated. This is appropriate for an academic project but is the most significant gap between the current system and a production-deployable application.

11.1.2 Client-Side Session Management

User sessions are managed entirely through localStorage on the client side. There is no server-side session token, HTTP-only cookie, or JSON Web Token (JWT) issued upon authentication. This means that a technically sophisticated user could modify their localStorage to impersonate another user or escalate their own perceived identity. While this is unlikely in an academic context, it is a fundamental security limitation that must be addressed before production deployment.

11.1.3 Static Seat Availability

The set of unavailable (taken) seats is defined as a static constant (TAKEN) in booking.js. It does not query the database to determine which seats have actually been booked for a given film, date, and showtime combination. In a real-world booking system, multiple users booking the same film at the same time could both see and select the same seats, leading to overbooking. Resolving this requires real-time seat availability queries and server-side locking mechanisms.

11.1.4 No Email Confirmation

The system does not send email confirmations or e-tickets to users after a successful booking. Users receive a booking summary in the success modal, but this information is not delivered to their email inbox. For a real cinema booking system, email confirmation is an essential feature that users expect.

11.1.5 No Administrative Interface

There is no administrative dashboard for theatre operators. Theatre managers cannot add new films, edit film details, manage showtimes, view booking reports, or perform any administrative actions through the application interface. All database modifications require direct access to MySQL (via phpMyAdmin or SQL commands).

11.1.6 No Password Recovery

The system does not implement a 'Forgot Password' or password reset workflow. Users who forget their password have no way to recover access to their account through the application interface. Account recovery would require direct database manipulation by an administrator.

11.1.7 No Real-Time Seat Locking

When two users simultaneously view the seat map for the same film and showtime, they see the same seats as available. Without real-time seat locking (using WebSockets or polling), both users could select the same seats and complete their bookings, resulting in a conflict. Resolving this requires a seat reservation system that holds selected seats exclusively for a user for a limited time (e.g., 10 minutes) while they complete the booking wizard.

11.2 Future Enhancements

The following enhancements are presented in priority order based on their importance to production readiness.

19. Payment Gateway Integration: Integration with Razorpay (for India), Stripe (international), or PayPal would transform the demonstration payment form into a functional transaction processor. The Razorpay JavaScript SDK provides a ready-

to-use checkout modal that can be initiated client-side with an order ID generated by the PHP backend.

20. Real-Time Seat Availability: Replace the static TAKEN constant with a dynamic query that fetches occupied seats for the selected film, date, and showtime from the bookings table before rendering the seat map. Add a temporary seat reservation table to the database to hold seats locked by in-progress booking sessions.
21. Email Confirmation and E-Tickets: Integrate PHPMailer or the SendGrid API to send HTML email confirmations including a formatted e-ticket (with booking reference, QR code, film details, seat numbers) to the user's email address upon successful booking.
22. Administrative Dashboard: Develop a separate admin panel (admin.html + admin.js + admin.php) with authentication separate from the user system (admin-specific login with role-based access). The dashboard should allow: adding and editing films, managing showtimes, viewing booking reports with filtering and export, and manual booking management.
23. Secure Server-Side Sessions: Replace localStorage-based session management with PHP sessions (session_start(), $_SESSION) or stateless JWT tokens issued upon login and validated on each API request. This eliminates client-side session tampering risks.
24. Password Recovery: Implement a forgot-password flow using a time-limited, single-use reset token. When a user submits their email to the reset form, the system generates a UUID token, stores it in a password_resets table with an expiry timestamp, and sends a reset link by email. Following the link opens a form where the user can set a new password.
25. Progressive Web App (PWA) Support: Add a Web App Manifest file and a Service Worker to enable home screen installation on mobile devices, offline browsing of the film catalogue, and background sync for booking submission when connectivity is restored.
26. Multi-Language Support: Implement internationalisation (i18n) using a simple JSON-based translation dictionary to support regional Indian languages (Tamil, Telugu, Hindi) in addition to English, improving accessibility for non-English-speaking audiences.

27. User Profile Management: Add a user profile page where authenticated users can update their name, email, and password, and set preferences such as preferred cinema location and default showtime.
28. Analytics Integration: Integrate Google Analytics or a self-hosted analytics solution (e.g., Plausible) to track page views, booking funnel drop-off rates, popular films, and peak booking times, providing actionable insights for theatre management.

11.3 UI Screen Shorts

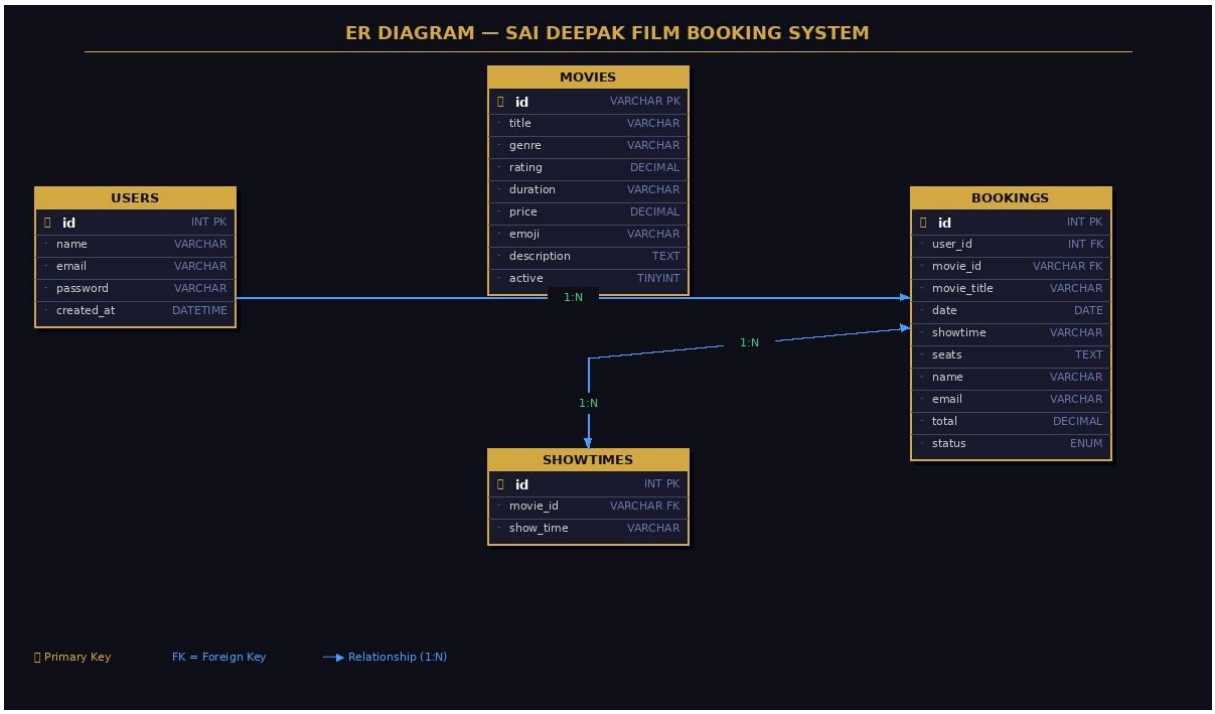

Figure 1: Entity-Relationship Diagram — Sai Deepak Film Booking System

Figure 2: Entity Relationship Diagram

Home Page — Hero & Movie Grid

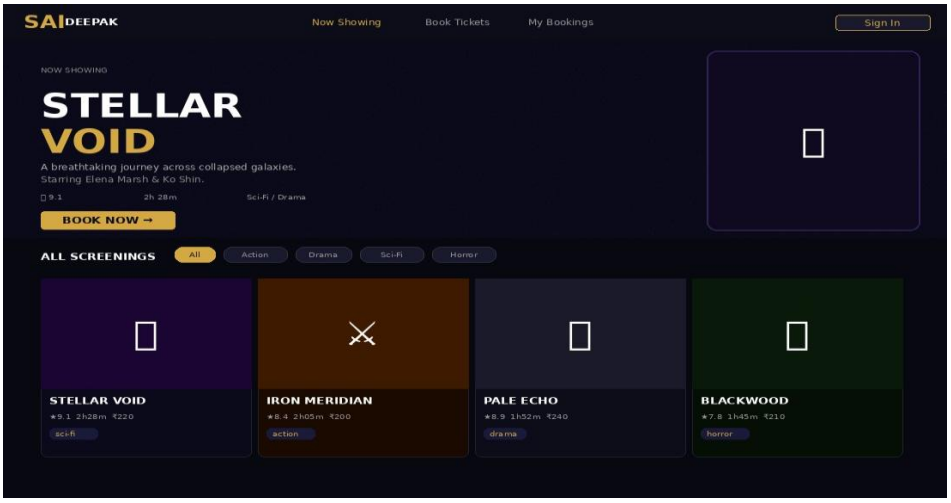

Figure 2: Home Page — Hero Section & All Screenings Movie Grid Booking

Page — Step 1: Film & Showtime Selection

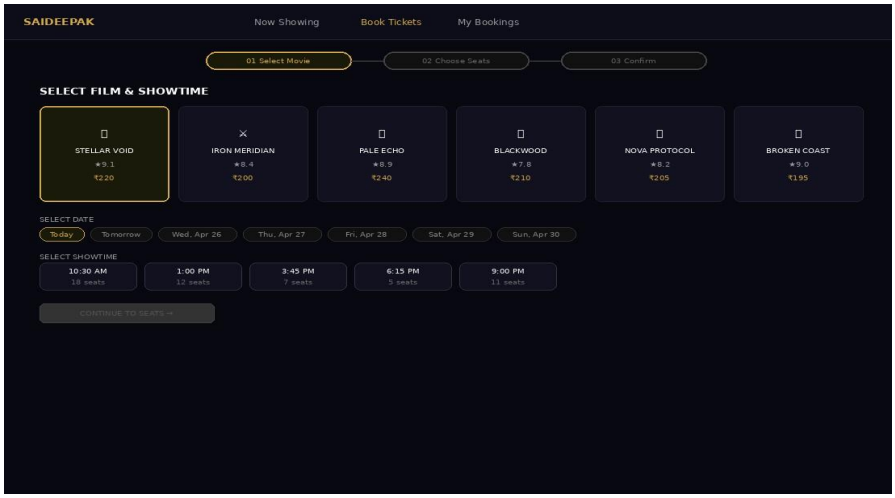

Figure 3: Booking Page — Step 1: Film & Showtime Selection

Booking Page — Step 1: Film & Showtime Selection

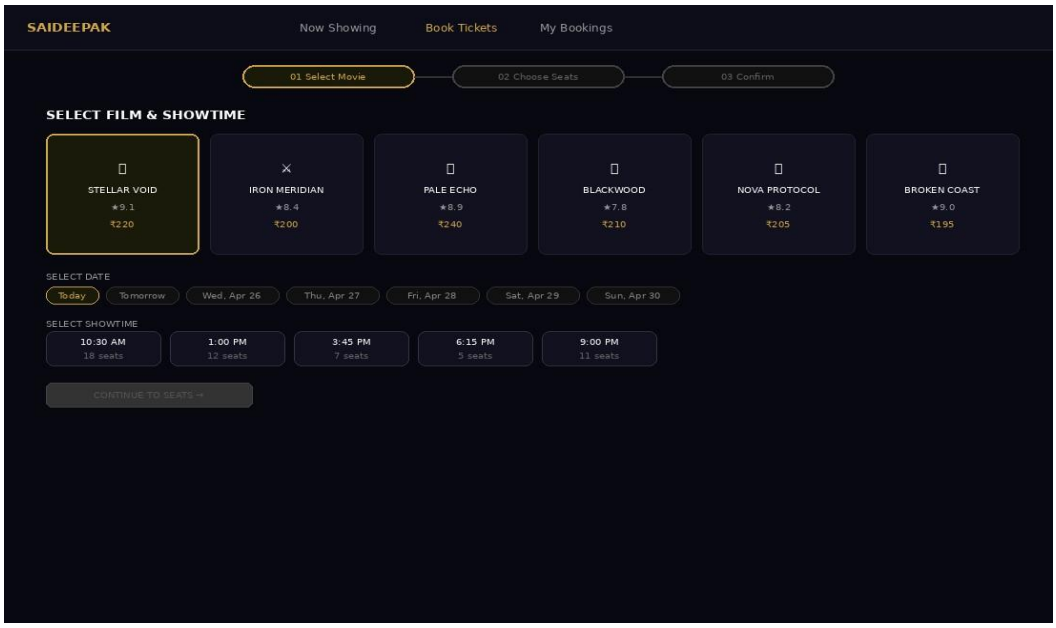

Figure 3: Booking Page — Step 1: Film & Showtime Selection

Booking Page — Step 2: Interactive Seat Map

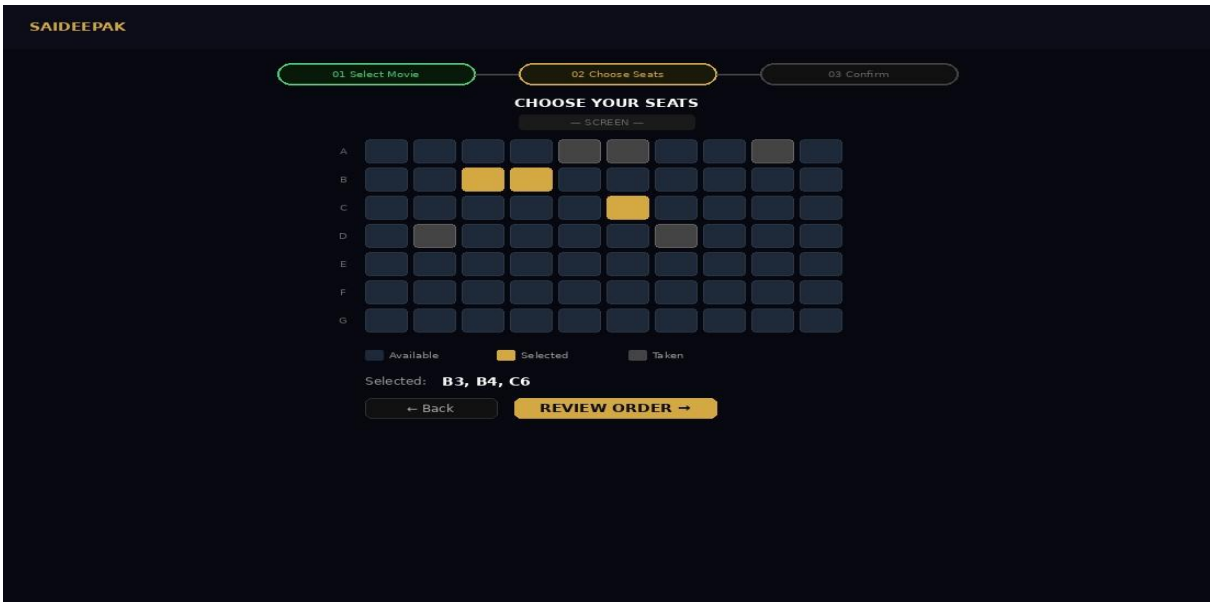

Figure 4: Booking Page — Step 2: Interactive Seat Selection Map

Booking Page — Step 3: Order Confirmation & Payment

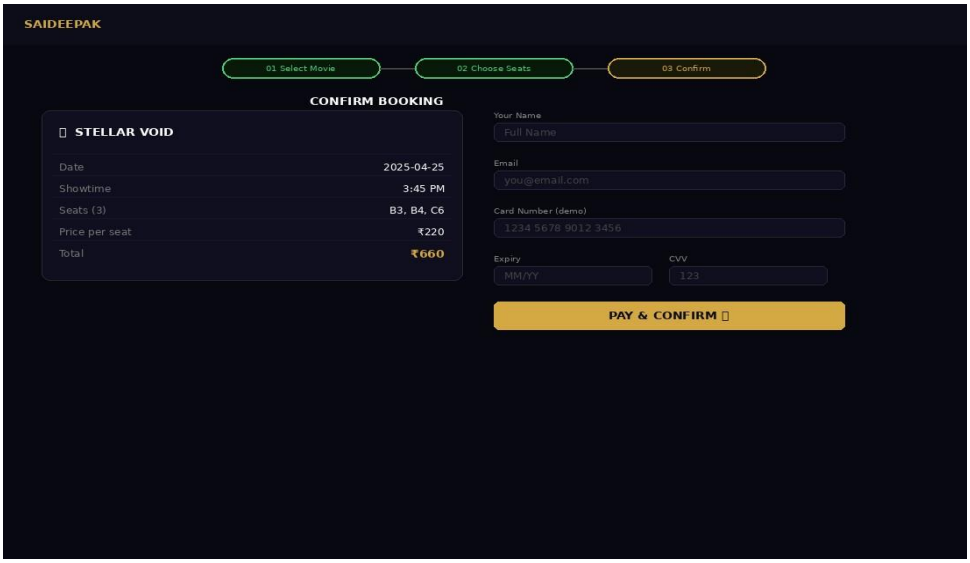

Figure 5: Booking Page — Step 3: Order Confirmation & Payment Form Booking

Confirmation — Success Modal

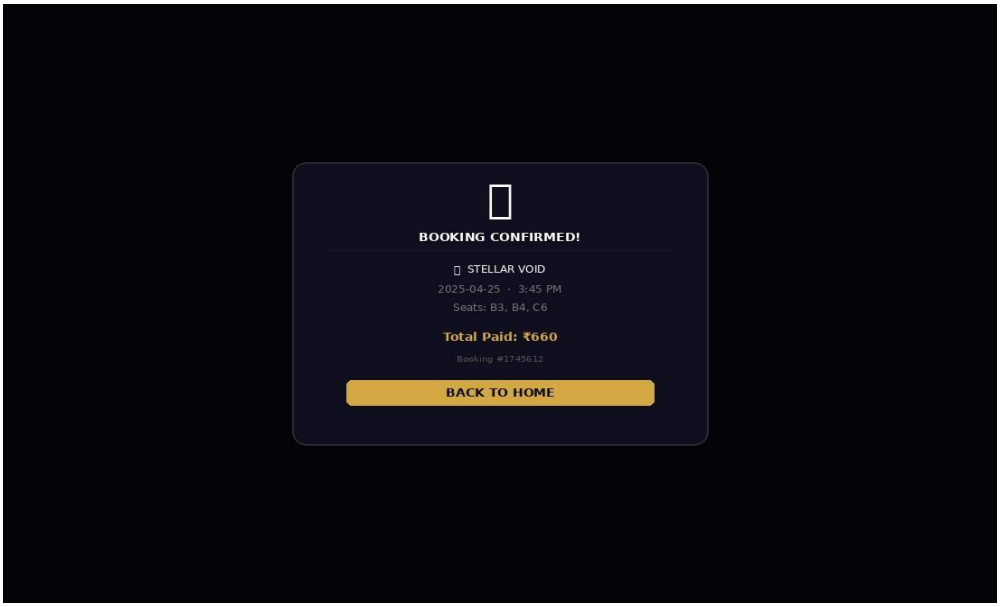

Figure 6: Booking Confirmation — Success Modal with Booking Summary

My Bookings — Booking History

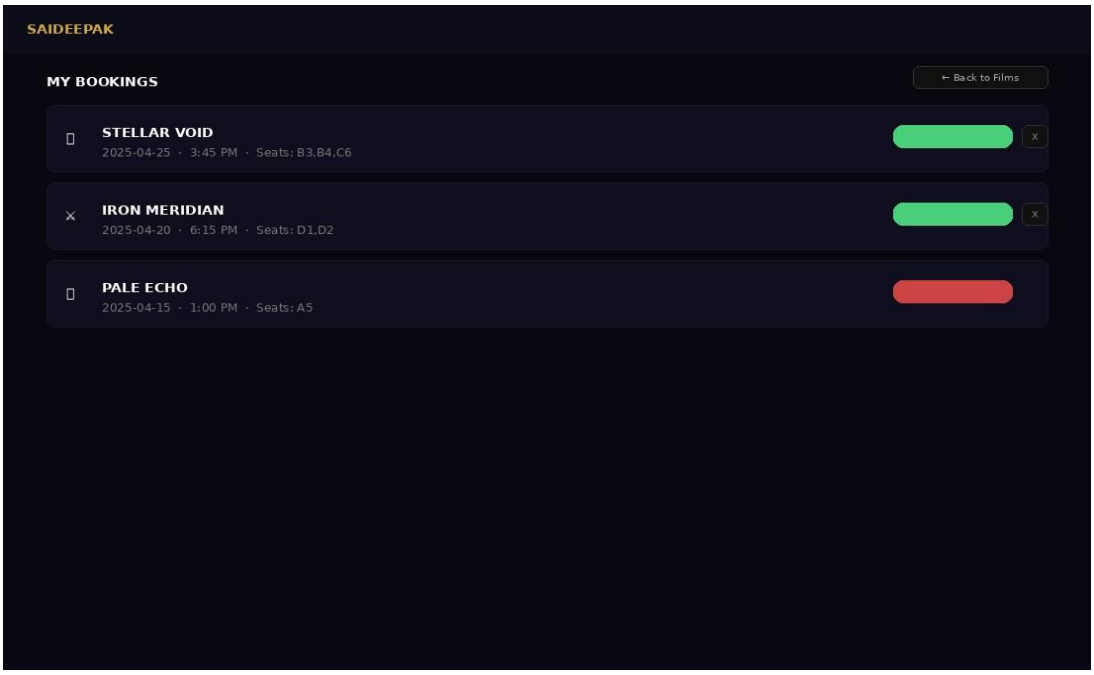

Figure 7: My Bookings — User Booking History with Cancel Option

12. CONCLUSION

The Sai Deepak Film Booking System successfully demonstrates the development of a complete, end-to-end cinema ticket reservation application using standard, widely-supported web technologies. The project meets all stated objectives and covers all essential aspects of a real-world booking system: user authentication with secure password handling, a comprehensive film catalogue with genre filtering, an interactive seat selection interface, a streamlined three-step booking wizard, payment capture with form validation, and booking history management with cancellation support.

The architectural design of the system is notable for its dual-mode operation strategy — the ability to function fully in both online full-stack mode (PHP/MySQL) and offline localStorage mode. This design decision reflects a sophisticated understanding of the real-world constraints of demonstrating and deploying web applications in diverse environments. It also demonstrates the value of clean abstraction boundaries: by wrapping all data operations in the apiGet() and apiPost() functions, the higher-level workflow code remains entirely decoupled from the data persistence mechanism.

The cinematic dark-theme user interface provides a professional, immersive experience that is consistent with the aesthetic standards of contemporary entertainment platforms. The careful application of Bebas Neue display typography, gold accent colours, CSS grain overlays, and smooth transition animations creates a cohesive visual identity that elevates the project beyond a functional exercise into a genuinely polished product prototype.

The three-step booking wizard architecture guides users through the reservation process in a logical, intuitive manner, with clear visual feedback at every stage. The step indicator, seat map, date/time picker, and order summary card work together to create a booking experience that is both informative and efficient.

The project demonstrates competence across a broad range of full-stack web development skills: frontend development with HTML5, CSS3, and vanilla JavaScript; backend development with PHP and MySQLi; relational database design with MySQL including normalised table schemas and foreign key constraints; REST API design with

JSON request/response formatting and prepared statement security; and responsive user interface construction with CSS Grid, Flexbox, and media queries.

The limitations identified in Chapter 11 — particularly the demonstration-only payment module, client-side session management, and static seat availability — are appropriate for the academic scope of the project and do not diminish its value as a learning exercise or prototype. The future enhancement roadmap presented in the same chapter demonstrates a clear understanding of what would be required to transform the prototype into a production-ready application.

In conclusion, the Sai Deepak Film Booking System represents a comprehensive, well-designed, and functionally complete cinema booking prototype. It provides a solid and extensible foundation upon which real-world enhancements — including payment gateway integration, real-time seat management, administrative tools, and security hardening — can be confidently built. The project fulfils its educational objectives while simultaneously producing a deliverable of genuine practical value.

13. REFERENCES

The following references were consulted during the development and documentation of the Sai Deepak Film Booking System.

13.1 Official Documentation

29. Mozilla Developer Network (MDN). HTML5 Reference and Guides. Available at: https://developer.mozilla.org/en-US/docs/Web/HTML
30. Mozilla Developer Network (MDN). CSS3 Reference and Guides. Available at: https://developer.mozilla.org/en-US/docs/Web/CSS
31. Mozilla Developer Network (MDN). JavaScript Guide and Reference (ES6+). Available at: https://developer.mozilla.org/en-US/docs/Web/JavaScript
32. Mozilla Developer Network (MDN). Fetch API Documentation. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
33. Mozilla Developer Network (MDN). Web Storage API (localStorage). Available at: https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API
34. PHP Group. PHP Manual — Version 7.4. Available at: https://www.php.net/manual/en/
35. PHP Group. password_hash() Function Reference. Available at: https://www.php.net/manual/en/function.password-hash.php
36. MySQL Documentation. MySQL 5.7 Reference Manual. Available at: https://dev.mysql.com/doc/refman/5.7/en/
37. MySQL Documentation. InnoDB Storage Engine. Available at: https://dev.mysql.com/doc/refman/5.7/en/innodb-storage-engine.html

13.2 Web Design and Typography Resources

38. Google Fonts. Bebas Neue Typeface by Dharma Type. Available at: https://fonts.google.com/specimen/Bebas+Neue
39. Google Fonts. Barlow Typeface by Jeremy Tribby. Available at: https://fonts.google.com/specimen/Barlow
40. CSS-Tricks. A Complete Guide to CSS Grid. Available at: https://css-tricks.com/snippets/css/complete-guide-grid/

41. CSS-Tricks. A Complete Guide to Flexbox. Available at: https://css-tricks.com/snippets/css/a-guide-to-flexbox/

13.3 Development Tools

42. diagrams.net (draw.io). Free Online Diagram Software for ER Diagrams. Available at: https://app.diagrams.net/
43. XAMPP. Cross-platform Apache, MySQL, PHP, and Perl Distribution. Available at: https://www.apachefriends.org/
44. Visual Studio Code. Source Code Editor by Microsoft. Available at: https://code.visualstudio.com/
45. phpMyAdmin. MySQL Database Management Interface. Available at: https://www.phpmyadmin.net

13.4 Academic and Technical References

46. Duckett, J. (2011). HTML and CSS: Design and Build Websites. John Wiley & Sons.
47. Duckett, J. (2014). JavaScript and JQuery: Interactive Front-End Web Development. John Wiley & Sons.
48. Nixon, R. (2018). Learning PHP, MySQL & JavaScript. O'Reilly Media.
49. Freeman, E., & Robson, E. (2014). Head First HTML and CSS. O'Reilly Media.
50. Welling, L., & Thomson, L. (2016). PHP and MySQL Web Development (5th ed.). Addison-Wesley Professional.

— End of Document —